



Beyond RAG: Advanced Agentic Knowledge Architectures

Week 3: Chunking

Summary

SupportVectors AI Lab

Copyright Notice

© SupportVectors. All rights reserved.

This document is the intellectual property of SupportVectors, and part of its training material. Only the participants in SupportVectors workshops are allowed to study the document for educational purposes currently, but is prohibited from copying or using it for any other purposes without written permission.

Contents

1	Foundations of Knowledge Architectures	4
1.1	The Fundamental Question: Why?	4
1.1.1	Primary Motivations	5
1.1.2	The Deceptive Promise of the Infinite Context Window	5
1.1.3	Attention Dilution and the Credit Card Analogy	5
1.1.4	Context Rot and the Density of Ambiguity	6
1.2	Granularity and Context Engineering	6
1.2.1	Point Queries vs. Evaluative Queries	6
1.2.2	The Library of Congress Paradigm	7
1.2.3	The Knapsack Problem	7
2	The Epistemology of Chunking	8
2.1	Too Large Chunks: Linearization and The Centroid Delusion	8
2.2	Arbitrary or Too Small Chunks	9
2.2.1	Endophora: Severing Grammatical Anchors	9
2.2.2	Rhetorical Structure Theory and Bridging Inferences	10
2.2.3	The Negation Trap and Scope Failure	10
2.2.4	Presupposition Accommodation	10
3	Getting Real: The Fallacy of Uniform Chunking	11
3.1	Mixing Tribes and Private Languages	11
3.2	The 512-Token Fallacy	12
3.3	Connective Tissue and Anisotropy	12
3.4	The Fragility of Semantic Meaning	13
4	The Fundamental Challenge	14
4.1	The Half-a-Worm Problem	14
4.2	The Endophora Problem	14
5	Text RAG vs. Visual RAG	15
5.1	The Visual Page as a Semantic Unit	15
5.2	The PRML Experiment	16
5.3	Determining Which Approach Works Best	17
5.4	Index Creation and Multiple Approaches	17
6	Basic Chunking Strategies	18
6.1	Fixed-Size Chunking	18
6.2	Sentence and Paragraph Boundary Chunking	18
6.3	Recursive Chunking	19
6.4	Markdown/Structural Chunking	19

6.5	Hierarchical Chunking	19
7	Advanced Chunking Methods: Semantic Approaches	20
7.1	LLM-Based Semantic Chunking	20
7.2	Embedding-Based Semantic Chunking	21
8	State-of-the-Art Chunking Methods	22
8.1	Contextual Chunking	22
8.1.1	The Paradox	22
8.1.2	The Solution: Contextual Preface Enrichment	22
8.1.3	Optimizing Compute: Determining What Context to Provide . .	23
8.2	Late Chunking	24
8.2.1	Core Insight	24
8.2.2	Why It Works	25
9	Shifting Paradigms	26
9.1	Visual Embeddings	26
9.1.1	The Concept	26
9.1.2	When Visual Embedding Works Best	26
9.1.3	The DeepSeek OCR Paper	26
9.1.4	Vision Language Model Demonstration	27
9.1.5	Visual Chunking for Video	27
10	Key Takeaways	28

“
The meaning of a word is its use in the language.
 ”

Ludwig Wittgenstein

(1889–1951), Austrian-British philosopher whose conceptualization of language games parallels the contextual mechanisms of modern language models.

1 Foundations of Knowledge Architectures

Building effective knowledge architectures demands a geometric and probabilistic understanding of how machines reason about text. A geometric approach frames textual meaning within an embedding space, allowing systems to quantify semantic relationships. Central to this architecture is the softmax function, which governs token probabilities, alongside foundational models that derive reasoning capabilities through multi-lingual structural games involving languages such as Tibetan, Spanish, and English. The core mechanisms of self-attention are derived from these first principles, integrating advanced cognitive mappings like butterfly prompt decomposition to orchestrate complex reasoning.

A critical mechanism for refining these embeddings is the application of contrastive loss. When a machine is trained using contrastive loss, it is guided by two opposing geometric forces: a force of attraction between semantically similar items and a force of repulsion between dissimilar items. This dual-force mechanism directly mitigates anisotropy, the tendency for embedding vectors to cluster narrowly. By reducing anisotropy, the embedding space achieves a better spread and superior utilization of the hypersphere’s surface. Different subjects float away from each other, resulting in a more diffused, highly separated structure where the compactness of similar concepts is precisely controlled by a mathematical temperature parameter.

1.1 The Fundamental Question: Why?

In any advanced retrieval system, the foundational step involves taking a raw document, parsing it into a structured format such as Markdown or plain text, and breaking it down

into smaller, distinct segments. While this process, known as chunking, is widely utilized, its underlying epistemology requires intense scrutiny. It is necessary to ask fundamental questions: Why must documents be chunked? What are the structural consequences of this division? Should the process be bypassed entirely in favor of emerging capabilities?

1.1.1 Primary Motivations

One core motivation for this division originates from principles of software engineering. Just as programmers utilize hashing functions to differentiate discrete objects within a system, chunking serves to separate textual concepts so they can be isolated, encoded, and retrieved independently. Beyond cognitive structuring, a primary consideration revolves around engineering constraints. Cloud computing providers charge based on token input, creating a direct financial cost for processing massive documents. Furthermore, processing vast amounts of text incurs substantial latency. While hardware-level mechanisms like KV caching, which stores precomputed keys and values in the attention layers, partially mitigate the quadratic compute costs during query response, the bandwidth required to read and transport massive datasets across networks remains a severe bottleneck. However, the deepest flaws in processing unchunked documents stem not from engineering limitations, but from cognitive and mathematical degradation within the model itself.

1.1.2 The Deceptive Promise of the Infinite Context Window

A prevalent argument in modern technology discourse is that Retrieval-Augmented Generation is obsolete due to the advent of frontier models possessing massive context windows capable of holding one to two million tokens. A cursory survey of video platforms yields a flood of content declaring that “RAG is dead,” asserting that these vast context windows have rendered retrieval architectures completely unnecessary. The assumption posits that if an entire document fits within the context window, parsing and chunking are unnecessary. This assumption is mathematically and practically flawed.

1.1.3 Attention Dilution and the Credit Card Analogy

In attention-based language models, every single token computes an attention weight relative to every other token. To understand the consequences of this, consider the anatomy of a monthly credit card bill. At the end of a billing cycle, a massive total balance is rarely the result of a single, catastrophic purchase. Instead, the balance is accumulated through dozens of trifling, low-value micro-transactions, a coffee, a bus ticket, routine library dues.

The softmax function within self-attention mechanisms operates identically. When evaluating a specific, high-value token such as *bank* in the sentence *She swam across the river to the other bank*, the model must assign heavy attention weights to highly

pertinent words like *river*, *she*, and *across*. However, the mathematical necessity of the softmax equation dictates that the model must also assign fractional attention to every other word in the provided text. When an unchunked, massive document is fed into the context window, the model's finite attention capacity is squandered on tens of thousands of irrelevant, trifling tokens. This attention dilution drastically reduces the model's ability to focus on the strictly pertinent evidence.

1.1.4 Context Rot and the Density of Ambiguity

Human language possesses an inherent density of ambiguity. Practical wisdom dictates that when speaking to legal authorities, one should be exceptionally brief, answering only the precise question asked. Volunteering excess information introduces unnecessary ambiguity, potentially framing guilt through contradictory statements regardless of actual innocence.

Providing a language model with an entire, massive document introduces the exact same vulnerability. Expanding the text volume expands the surface area for contradictions, irrelevant tangents, and contextual rot. Historically, this manifested as the *Lost in the Middle* phenomenon. Human readers naturally remember the opening lines of a text, such as the famous *Hello World* introduction in the classic *The C Programming Language* textbook [Kernighan and Ritchie \[1988\]](#), and the climactic unraveling of a mystery novel's ending. However, the contents of page 100 are invariably forgotten. Language models exhibit an identical degradation, severely struggling to retrieve precise facts buried deep within a bloated, unchunked context window.

1.2 Granularity and Context Engineering

Determining exactly what information to feed an intelligent system requires distinguishing between the fundamental types of user queries and understanding the physical limits of enterprise data.

1.2.1 Point Queries vs. Evaluative Queries

Queries exist on a spectrum of granularity. A point query demands a strictly atomic fact: *What is the capital of Germany, and what is its population?* If a user requires this specific data, handing them all 26 volumes of the Encyclopedia Britannica is actively unhelpful. Common sense dictates locating the exact volume, chapter, and paragraph detailing Berlin's demographics and providing only that specific, high-fidelity snippet.

Conversely, evaluative queries contain aleatoric uncertainty, a fundamental ambiguity regarding the required granularity of the answer. Such queries require sensemaking over an entire corpus. Consider a narrative adaptation of Dostoevsky's *Crime and Punishment* [Dostoevsky \[1866\]](#), modernized to feature a CEO of a high-flying AI company, one

actively building advanced generative models, distinctly not relying on legacy algorithms like Support Vector Machines. The CEO secures massive funding by fabricating revenue and misrepresenting total addressable markets to investors. For years, he avoids detection but lives in a state of paralyzing psychological torture, waking up terrified every day. When the authorities finally close in and he confesses, he is thrown into prison, and paradoxically experiences his first good night's sleep in years.

If a system is asked, *What was the punishment for his crime?*, a coarse extraction might answer that the punishment was a multi-year prison sentence. However, a true evaluative analysis requires absorbing the entire narrative to deduce that the actual punishment was the preceding years of anxiety and psychological suffering, and the prison sentence was simply the release. While evaluative queries seem to demand the ingestion of entire documents, the reality of data scaling makes this impossible.

1.2.2 The Library of Congress Paradigm

Humanity's data volume is currently experiencing explosive, exponential growth, doubling approximately every two years. While frontier research, such as DeepSeek's Engram (EnM) paper [DeepSeek-AI \[2026\]](#), explores the possibility of utilizing extra memory as a sidecar to inject facts directly into a model without expanding the base window, the underlying hardware capabilities and primary context windows still scale at a sub-exponential, linear rate. It is physically impossible to fit an entire enterprise corpus, analogous to the Library of Congress, into any single context window. Because a complete corpus cannot be ingested simultaneously, retrieval architectures must extract and supply highly specific subsets of that data.

1.2.3 The Knapsack Problem

The context window functions as a highly constrained knapsack. If a user asks a complex question regarding the Maxwell-Boltzmann distribution, the system cannot load an entire thermodynamics library into the knapsack. It must identify and extract only the most precise, pertinent paragraphs across multiple physics texts.

This process is the essence of context engineering. From a Bayesian perspective, the language model approaches a query with a baseline prior belief. The curated, high-quality facts placed within the knapsack serve as the critical evidence. Synthesizing this injected evidence with the model's intrinsic capabilities produces the most accurate posterior output.

Questions for Pondering

- How does the application of contrastive loss resolve the issue of anisotropy within an embedding space?
- What vulnerability is introduced when feeding a language model an entire, massive document at once?

2 The Epistemology of Chunking

If the context window must be curated with extreme precision, the central architectural challenge becomes identifying the exact boundary, or size, of a thought.

2.1 Too Large Chunks: Linearization and The Centroid Delusion

Written text inherently forces complex, multi-dimensional concepts into a linearized, sequential format. An author chains Thought A to Thought B, progresses to Thought C, and may later loop back to a related Thought A'.

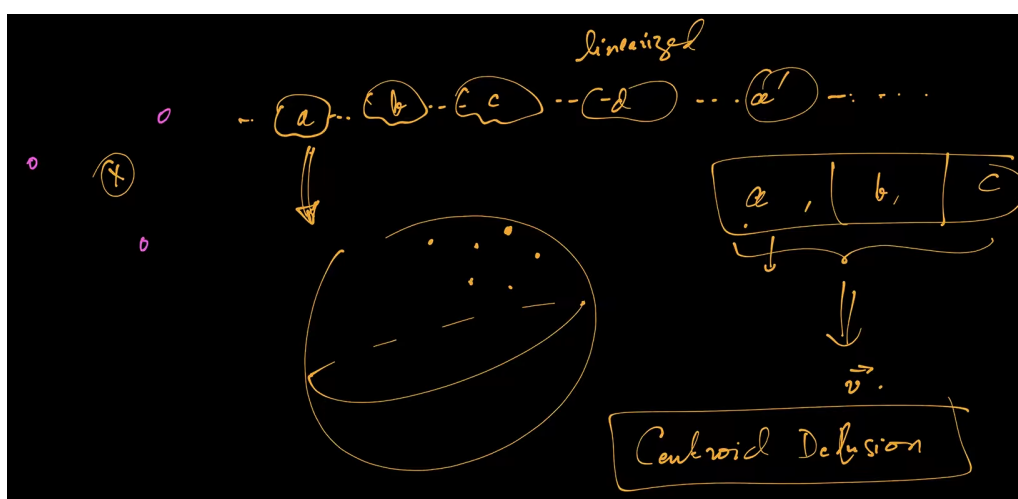


Figure 1: Illustration of Linearization and Centroid Delusion

If distinct thoughts map to pure, specific points on the surface of an embedding hypersphere, applying arbitrary, block-based chunking creates severe mathematical distortions. Consider a single text chunk containing three completely unrelated facts:

1. The cow jumped over the moon.
2. Nvidia stocks crashed today.
3. Today is a good day to go to the beach.

Embedding each sentence individually yields three highly distinct vector coordinates. However, concatenating these strings into a single chunk and passing it through an embedding model forces the system to calculate an arithmetic centroid location between the three points. The resulting embedding vector represents an averaged, middle ground that fails to accurately capture the semantic truth of any individual thought. This mathematical homogenization is known as the **Centroid Delusion**, rendering the resulting vector utterly useless for precise semantic retrieval.

2.2 Arbitrary or Too Small Chunks

Attempting to avoid the Centroid Delusion by creating highly granular, small chunks introduces devastating linguistic fragmentation. Human language relies on invisible structural dependencies that break down entirely when text is blindly sliced.

2.2.1 Endophora: Severing Grammatical Anchors

Consider the sequence: *Sally swam across the river. Then she ate her breakfast.*

If a chunk boundary severs these two sentences, the resulting isolated chunk, *Then she ate her breakfast*, loses all semantic meaning. The pronoun *she* relies on an **anaphoric** relationship, pointing backward to the anchor entity *Sally*. Alternatively, **cataphora** occurs when a pronoun points forward to an entity introduced later in the text (*She swam... then Sally ate*). Together, these mechanisms are known as **Endophora**.

When naive chunking algorithms blindly slice text, they routinely decapitate these endophoric links. Similarly, textbooks frequently reference external anchors (for example: *Read this in conjunction with Equation 3.6*), and academic papers point to bibliographies at the far end of the document. Isolating these references dilutes the contextual fabric essential for robust self-attention.

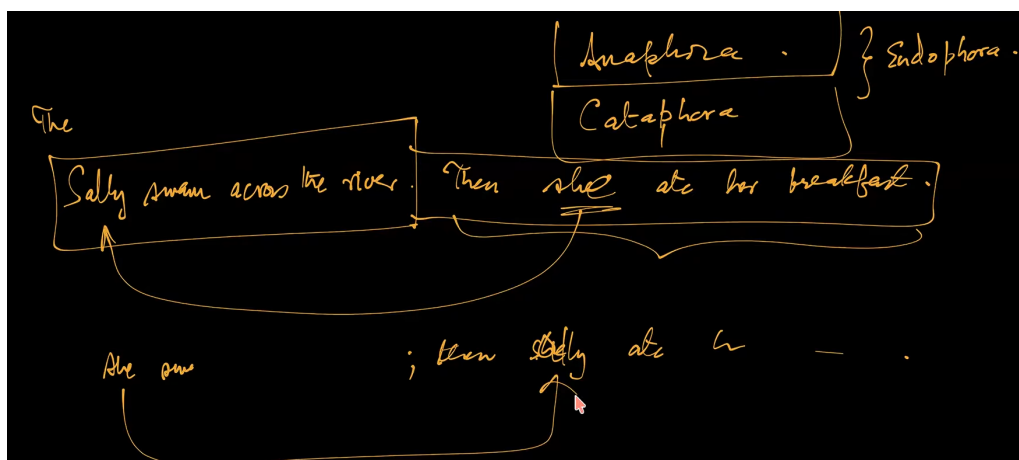


Figure 2: Illustration of Endophora.

2.2.2 Rhetorical Structure Theory and Bridging Inferences

Documents are not mere sequences of characters; they are complex, interlocking webs of relationships governed by Rhetorical Structure Theory. Sentences rely on unstated, implicit logic known as bridging inferences. For instance: *John walked into a room. The chandelier was magnificent.* The text never explicitly states that the chandelier was located on the ceiling of that specific room, yet the human mind instantly builds that logical bridge. Separating these sentences via chunking destroys the capacity for models to infer that unstated spatial relationship.

2.2.3 The Negation Trap and Scope Failure

Scope failures represent the most dangerous side effect of arbitrary chunking because they do not simply lose information, they actively invert the truth. This occurs frequently when cause-and-effect relationships or critical negations span across a text boundary.

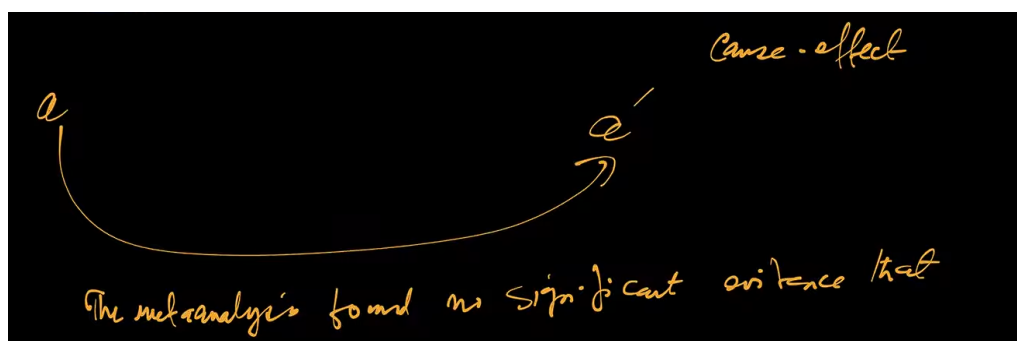


Figure 3: Illustration of cause-and-effect relationships within text.

Consider a highly contextualized finding within a medical document:

The meta-analysis found no significant evidence that Compound X increases cardiac risk in patients under 65. However, in patients 65-80 with hypertension, the data suggested a concerning elevation.

If a retrieval algorithm splits this paragraph and surfaces only the second sentence, it presents a stark, terrifying medical alarm. Conversely, if it surfaces only the first sentence, all warnings vanish. The absolute scientific truth exists solely within the relationship between the two sentences. Cutting the text at the boundary of that relationship actively fabricates a falsehood.

2.2.4 Presupposition Accommodation

Academic and scientific writing frequently employs presupposition accommodation. A researcher will often begin a paper by outlining a highly tentative, heavily caveated hypothesis. However, to explore the logical boundaries of that hypothesis throughout the remainder of the paper, the researcher cannot constantly reiterate the hedging language.

SCOPE FAILURES DESERVE THE MOST ATTENTION because they do not merely lose information — they *invert* it. Negation is the most dangerous culprit. “The meta-analysis found *no* significant evidence that Compound X increases cardiac risk in patients under 65. However, in patients 65–80 with hypertension, the data suggested a concerning elevation.” Surface only the second sentence and the reassurance-with-a-caveat becomes pure alarm; surface only the first and the caveat vanishes. Neither is the truth. The truth is the *relation* between them — and that relation lives exactly at the boundary where your chunker cuts.

Figure 4: A highly contextualized text.

By academic convention, the author proceeds to write about the hypothesis as though it were a proven fact.

If a chunking system extracts a paragraph from the latter half of the paper, the isolated text will convince the language model that the concept is a universally established scientific certainty, completely erasing the tentative framing established in the introduction.

Questions for Pondering

- What is the “Centroid Delusion,” and how does it compromise semantic retrieval when chunks are too large?
- How does arbitrary or overly small chunking break the linguistic concept of “Endophora”?

3 Getting Real: The Fallacy of Uniform Chunking

A fundamental structural failure preceding all parsing errors is **Genre Blindness**, the catastrophic assumption that all documents share an identical information architecture.

3.1 Mixing Tribes and Private Languages

Different academic and professional disciplines operate as distinct tribes, each utilizing a highly specialized private language. This tribal differentiation mirrors the ancient historical test of the shibboleth, where a group identified embedded spies by their inability to pronounce the specific word *shibboleth*, the impostors calling it *chibboleth* instead. Just as that single word separated people, modern textual tribes are separated by profound structural shifts. A computer science professional wandering into a chemistry department

will find themselves lost amidst discussions of molecular formulas, despite both groups speaking English. Beyond specialized vocabulary, the underlying rhetorical structure of the text shifts radically:

- **Novels** utilize narrative arcs, building tension, mystery, and climactic resolutions.
- **Legal Contracts** represent the exact antithesis of narrative. Documents like Section 5 of the European AI Act are written as hyper-dense, condensed code, designed to eliminate all ambiguity through rigid, precise clauses.
- **Scientific Papers** follow strict logical progressions, universally adhering to the IMRAD framework: Introduction, Methods, Results, and Discussion.

3.2 The 512-Token Fallacy

Despite these profound structural differences, the industry default for chunking text is frequently hardcoded to an arbitrary 512 tokens. Converting raw words to tokens generally follows a 4:3 ratio (accounting for prefixes, suffixes like *ing*, and punctuation marks). A standard printed line contains 8 to 10 words, equating to roughly 12 tokens. A standard page of 35 lines therefore contains approximately 420 tokens.

Setting a universal chunk size of 512 tokens equates to roughly one densely printed page. While educational environments or foundational clustering labs might implement arbitrary limits, such as mandating 300-token chunks for dense math and chemistry texts simply because adopting a naive, hardcoded threshold is an expedient starting point where a degree of procedural laziness is momentarily tolerated, deploying such rigid parameters in a production environment is architecturally indefensible. Assuming that exactly one pure, distinct thought exists within every single page of text, regardless of whether it is a fast-paced news article or a dense legal statute, is a profound architectural fallacy. One size does not fit all.

3.3 Connective Tissue and Anisotropy

When chunks are drawn too large, they inadvertently capture massive amounts of linguistic connective tissue, the high-frequency words and standard English syntax necessary to bind ideas together. While the core thoughts within two large chunks might be distinct, the sheer volume of common syntax dominates the vector embedding. The similarities drag the distinct points toward each other in the latent space, increasing anisotropy. This blurring of boundaries causes search algorithms to return generic, highly popular, but ultimately useless text segments rather than surgical, highly precise knowledge.

3.4 The Fragility of Semantic Meaning

The absolute necessity of intelligent, semantic text parsing over raw string manipulation is proven by the extreme fragility of human language. A shift in structural meaning does not require new vocabulary; it requires only the displacement of a single punctuation mark.

Consider the grammatical punctuation test evaluating the title of a well-known book:

1. Eats, shoots and leaves.
2. Eats shoots, and leaves.
3. A panda eats bamboo shoots and leaves.

The first sentence, populated with commas, outlines a rapid sequence of aggressive actions, instantly evoking the image of a gunslinger in a Western thriller. The second sentence, containing the exact same vocabulary but stripped of commas, transforms the verbs into nouns and direct objects, perfectly describing the dietary habits of a cute panda bear.

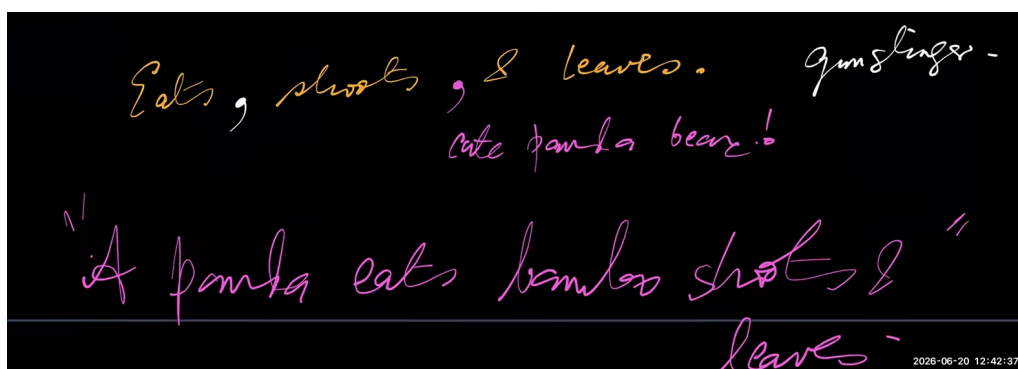


Figure 5: Illustration of the three sentences for grammatical punctuation exercise.

To empirically prove how critical this structural nuance is, one must calculate the cosine distances between these concepts within the embedding space. Evaluating the pairwise cosine similarities, $\cos(1,2)$, $\cos(1,3)$, and $\cos(2,3)$, is left as an exercise for the students.

Ideally, **(2)** and **(3)** should be near, and **(1)** should be far apart. However, when finding pair-wise correlations and pair-wise cosine similarities with embedding models on these sentences, almost all of them got it wrong. Quite often, the embedding models found that sentences one and two are much closer according to the model than sentences two and three. That is a disaster. **It shows that even today, June 2026, embedding models have to be used with caution.**

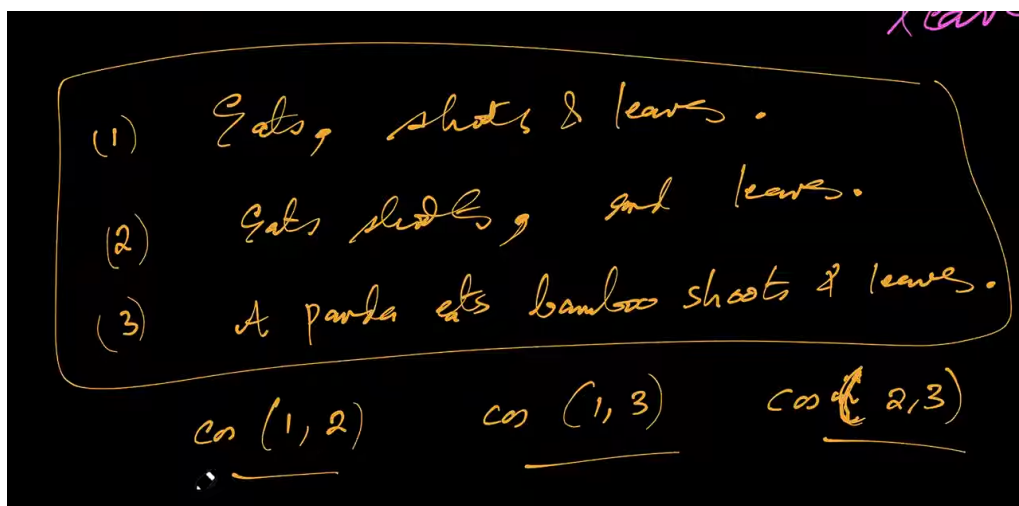


Figure 6: Illustration of the three sentences for grammatical punctuation exercise.

Questions for Pondering

- How do standard linguistic elements (connective tissue) increase anisotropy when chunks are drawn too large?
- Why does semantic meaning rely heavily on intelligent parsing rather than raw string manipulation?

4 The Fundamental Challenge

4.1 The Half-a-Worm Problem

When building RAG systems, the instinct of people is to segment documents. However, chunking inherently has problems and limitations. A classic analogy illustrates this problem: “What is worse than a worm in an apple? Half a worm.” When you bite into an apple and see half a worm there, you have about the same reaction as when you bite into documents and realize that it has split in the middle of a word or tokens. Fixed-size token chunking would be fixed-size string characters chunking (e.g., 512 characters, a thousand characters or something like that), but at least with tokens, you have something coming up. However, it might chunk in the middle of a sentence, which will completely destroy the semantic integrity of what was being said near the end or the beginning.

4.2 The Endophora Problem

Endophora represents a visual or textual nuance where text relies on other elements for its meaning, such as saying “as you see in the figure above,” “as you see on the table on the right,” or “as you see in the figure below.” When sentences are broken up blindly, this context is lost.

Revisiting our three sentence exercise demonstrates language nuances with three variations:

1. “Eats, shoots and leaves”
2. “Eats shoots, and leaves”
3. “A panda eats bamboo shoots and leaves”

Questions for Pondering

- What is the “Half-a-Worm Problem” and how does it relate to chunking?
- How does chunking exacerbate the Endophora problem?

5 Text RAG vs. Visual RAG

5.1 The Visual Page as a Semantic Unit

The structure of scientific textbooks in STEM fields is fundamentally multifaceted and multimodal. A single page is dwelling upon one thought and jointly talking of a single idea. A typical page contains four major components:

- Running text (prose)
- Figures/Images (e.g., an image of a coffee cup with steam/water vapors rising from it)
- Plots (e.g., a Maxwell-Boltzmann distribution plot where the y-axis is probability of energy and the x-axis is energy)
- Tables (e.g., a table of distributions’ mean, standard deviation, skewness, kurtosis, moments, etc.)
- The physical position and layout of these components on the page

The entire page has to be jointly studied as one single semantic unit. It would be wrong to say that the semantic content is just in the running prose. If you use a typical OCR tool that just parses this page and extracts only the text out of it, it would lose a lot. The extracted text gets out of context because the prose loses its context without the right positioning of figures. More and more technical documents conform to this layout structure rather than to running prose.

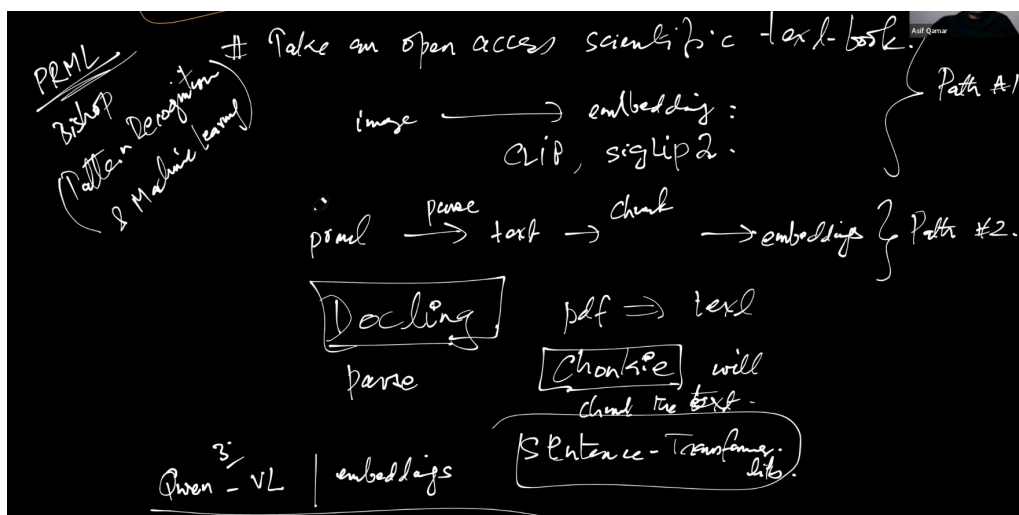


Figure 7: The figure illustrates the comparison between traditional text RAG and visual embedding approaches.

5.2 The PRML Experiment

A practical lab experiment was conducted using an open-access scientific textbook: *Pattern Recognition and Machine Learning* (PRML) by Bishop [Bishop \[2006\]](#), which has 758 pages. The experiment compared two unorthodox paths:

Traditional Text RAG Pipeline:

1. Convert the PDF using a text extraction parser such as **Docling**¹.
2. Get plain text out of it.
3. Chunkify it using **Chonkie**².
4. Create embedding vectors using the **Sentence Transformer**³ library.
5. Put the vectors into a vector DB, **Qdrant**⁴.

Visual Embedding Pipeline:

1. Chomp and chunkify the PDF strictly into images—each page becomes a PNG image (resulting in 758 PNGs).
2. Use a vision language understanding model, **Qwen 3**⁵, to create image embeddings.
3. Put the image embeddings into the **Qdrant** vector DB.

¹<https://github.com/DS4SD/docling>

²<https://github.com/bhavnicksm/chonkie>

³<https://sbnet.net/>

⁴<https://qdrant.tech/>

⁵<https://github.com/QwenLM/Qwen>

The two approaches showed distinct differences depending on the query:

- **Query 1:** A simple query asking “*What is Bayes theorem?*” accurately pointed straight to page 35 of the physical PDF, showing the famous Bayes theorem equation 1.12 (Chapter 1, Equation 12), as well as page 37 (marked as page 17 in the book due to table of contents offsets) talking about posterior probability.
- **Query 2:** A harder query asking “*Why does a ninth order polynomial fit the training points perfectly yet generalize poorly?*” strictly required understanding visual data. In machine learning, if you make a model that’s unusually flexible and overparameterized, it will memorize the data and overfit to the noise of the data instead of the underlying pattern, causing it to generalize poorly. The visual approach successfully surfaced page 28, which explicitly illustrated the overfitting problem, showing a plot where the test error shoots up at the ninth order polynomial ($M=9$) and a red line going through every single data point. It also surfaced page 31 talking about test/train error. This harder problem cannot be answered just by reading text, which is where traditional text embeddings get into trouble.

The Vector Database vs. LLM Distinction: When a vector DB is able to return the correct pages and answer questions, a question arises regarding the need for an LLM at the end of it. The vector DB does not actually answer the question; it merely gives the search results and points precisely to those pages which contain the answer. You strictly need the LLM at the end of the RAG pipeline to take those retrieved search results and actually write out a smart answer.

5.3 Determining Which Approach Works Best

Which approach works best depends entirely on the kind of input data. If the input has purely text, you can go for the text-based pipeline. If it has both text and images, you will have to try different things to see what works.

In the real world, a watered-down version of the “No Free Lunch Theorem” applies: a priori, you usually do not know which approach is best. You have to try all sorts of algorithms. Our field is entirely based on tournaments. To build a serious, world-class AI product instead of just applying cookie-cutter download libraries, you must run tournaments to see which branch wins.

5.4 Index Creation and Multiple Approaches

When building a RAG architecture, you can have a text-based approach and a visual approach. If you can afford to maintain both indices, you should keep both of them as search indices.

When you use the visual branch, a semantic embedding is the only choice you have. However, when you use the parser-chunker embedding approach, you have multiple possibilities available. You can use dense vectors (embedding vectors) or you can use sparse embeddings from basic keyword search engines of the past like ElasticSearch⁶, BM25, or Splade⁷.

Creating and fixing search indices is a one-time upfront capital cost. At inference time, searching one index, two indices, or three indices does not heavily matter because inference costs are low. When the quality of the result matters, you should maintain multiple indices for the same data. Splitting and indexing the same information in different ways across different search systems provides a massive performance boost. This introduces the problem of merging disparate search results together, which is a re-ranking problem. Wherever possible, run both indices because you cannot know a priori which will yield the right pages.

Questions for Pondering

- Why is visual embedding sometimes considered superior to traditional text parsing?
- What was the critical lesson from maintaining both text and visual index branches?

6 Basic Chunking Strategies

6.1 Fixed-Size Chunking

Fixed-size chunking treats an entire document string like a loaf of bread, going “chop chop chop” to create fixed slices of text. While 512 tokens is a highly popular choice in the industry, it is a bad practice. Choosing a size like 512 is too big and likely leads to centroid delusion problems, blurring, and a complete loss of semantic purity.

Fixed-size string character chunking (e.g., cutting exactly every 1,000 characters) is a disaster because it drops the knife and chunks right in the middle of words or sentences. This completely destroys the semantic integrity of what was being said near the beginning or the end of the chunk.

6.2 Sentence and Paragraph Boundary Chunking

A slightly smarter method is sentence boundary chunking, which cuts the text at the end of a sentence. For example, in Mandarin, sentences are structurally structured to

⁶<https://www.elastic.co/elasticsearch/>

⁷<https://github.com/naver/splade>

be semantically whole and form a complete unit, giving who, when, and where (e.g., “*I today in the restaurant ate dim sum*”). Breaking at a sentence boundary here looks good.

However, sentence boundary chunking fails when facing the problem of endophora across sentences, such as having a text say: “*This is true. What is truth?*” The context of what “this” refers to is entirely lost.

To fix this, you can move to paragraph boundary chunking to break text at paragraph limits. However, this approach is limited by varying author styles. For example, the historic British author Sir Francis Bacon had a penchant for writing incredibly long sentences, with one famous sentence starting at the top of a left-hand page and only finishing at the bottom of the right-hand page. While American English styles eschew long constructions for direct, short sentences, the legal community still uses highly complex sentence and paragraph structures. Furthermore, journalistic styles like the *New York Times* condense many different thoughts into single paragraphs because they know readers have short attention spans.

Therefore, paragraph boundaries also fail because an argument may pursue into the next paragraph, or the first paragraph may merely set up a premise that the second paragraph establishes. It is never completely clear where a thought begins and ends.

6.3 Recursive Chunking

Recursive chunking is a graceful degradation approach. If you give the system an approximate target size (like 100 or 512 tokens), the library will go to that size first and try to shorten it gracefully. It checks if it can end up with one whole paragraph; if not, it down-steps to see if it can at least end up with a complete sentence boundary. This recursive approach is the default choice in many open-source libraries and is a good baseline way to start.

6.4 Markdown/Structural Chunking

Markdown chunking breaks a document up structurally using headings, chapters, sections, and subsections. If a document has these structural elements available, you should definitely exploit them. When an author consciously starts a new section, they are implicitly stating: “*Okay, now let’s talk of something else, we are done with this, let’s move on to that.*” Structural elements are a God-given blessing from the author telling you exactly where the boundaries of a thought lie.

6.5 Hierarchical Chunking

Hierarchical chunking handles the dilemma where very short chunks have pure thoughts but suffer from endophora, while large chunks have no endophora issues but suffer from semantic impurity.

This strategy takes a hierarchical approach:

1. Start with a large parent chunk (e.g., 512 or 1,000 tokens) to capture a common broad theme (similar to how an entire page in Bishop's book is broadly about probability).
2. Subdivide that parent chunk into smaller child chunks (e.g., 50 to 70 tokens).

While industry practice often advocates using overlapping smaller chunks to avoid losing context, there is no strong or compelling evidence that text overlap makes a huge difference with endophora.

When storing hierarchical chunks, you maintain a metadata table tracking the lineage: which document/corporate corpus a chunk belongs to, and exactly which parent chunk a child belongs to. When a focused child chunk gets a nearest-neighbor hit during a query search, you have two architectural choices for what to send to the LLM:

- You can pick up all the immediate neighbors of that child chunk and send them together.
- You can completely bypass the small chunk and send the entire parent chunk (the whole page or big piece of text) to the LLM.

Questions for Pondering

- What is the primary limitation of fixed-size chunking algorithms?
- How does hierarchical chunking attempt to solve the fragmentation problem?

7 Advanced Chunking Methods: Semantic Approaches

7.1 LLM-Based Semantic Chunking

Instead of brute-force cutting, semantic chunking lets an AI read the text and identify the exact boundaries where a thought changes to cut the text cleanly. While a great concept, implementing it faces a “belling the cat” dilemma: all the mice agree that putting a bell on the cat makes them safe, but figuring out how to actually do it is the hard part.

You can implement this by providing a long paragraph of text to an LLM (like ChatGPT or Claude) with a prompt instructing it to break the text apart into semantically coherent chunks of a rough target size.

Small Model Optimization: Using a massive 3-trillion or 1-trillion parameter frontier model for chunking is an absolute waste of capacity. Small local models around the 1-billion to 2-billion parameter class have more than enough epistemic capacity to

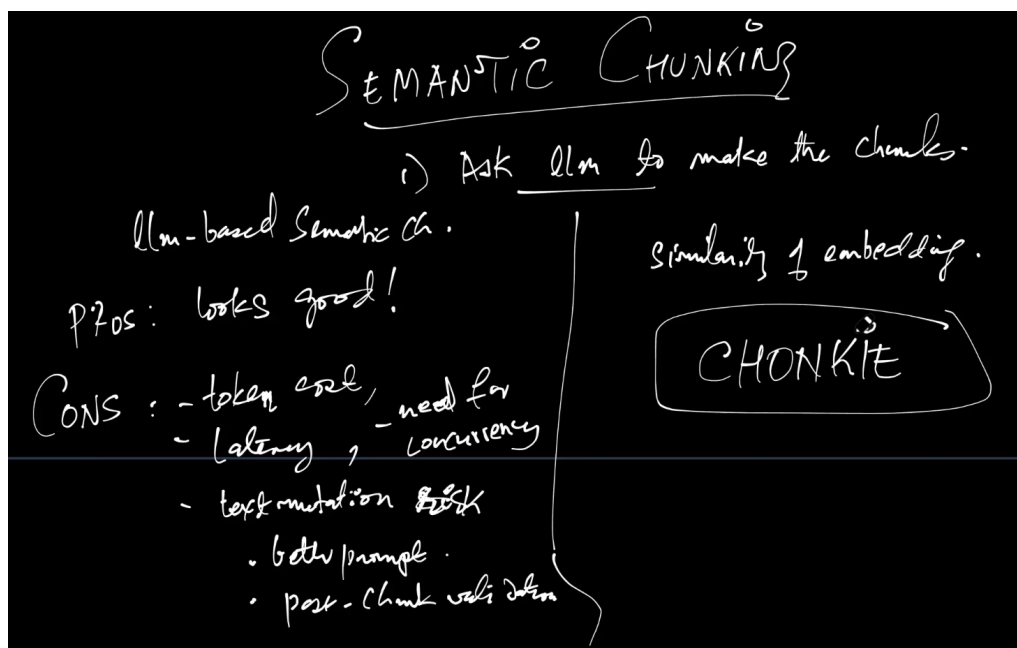


Figure 8: The figure illustrates the Semantic Chunking concept.

read text and find proper chunk boundaries. Models like Microsoft’s **Phi 4**⁸ series, Google’s **Gemma**⁹ series (like Gemma 3 or 4), or **Kimmy K2**¹⁰ do extraordinarily well at this task. Running these small models locally on a Ray cluster costs zero tokens, eliminates external API latency, and allows you to chomp through massive amounts of traffic efficiently.

7.2 Embedding-Based Semantic Chunking

This approach uses tiny encoder-style models to find semantic topic shifts geometrically. You take the first sentence of a text, embed it, and project it as a vector onto the hypersphere. You then embed the second sentence; if its vector lands close to the first, you group them together. As you process the third sentence, you monitor the geometric departure: how far is the new sentence vector from the previous sentence, or how far is it from the running average centroid of the previous vectors? The moment the vector distance departs past a set threshold, the system flags that a brand-new topic has started and cuts a chunk boundary.

Because encoder-style models require only a single forward pass per piece of text, they are incredibly tiny, cheap, and fast compared to LLMs. In the open-source community, a highly adopted library designed for managing this behavior is **Chonkie**.

⁸<https://huggingface.co/microsoft/phi-4>

⁹<https://ai.google.dev/gemma>

¹⁰<https://moonshot.cn/>

Questions for Pondering

- How does LLM-based semantic chunking differ from fixed-size chunking?
- Why are small local models preferred for semantic chunking over massive frontier models?

8 State-of-the-Art Chunking Methods

8.1 Contextual Chunking

8.1.1 The Paradox

Small chunks are ideal because they possess pure semantic thoughts, but they suffer from a massive loss of context and break endophora references. The paradox headline is: *How do you keep smaller chunks without losing context?* Summarizing paragraphs together changes the text itself, and grouping chunks together simply builds a larger, semantically impure chunk.

8.1.2 The Solution: Contextual Preface Enrichment

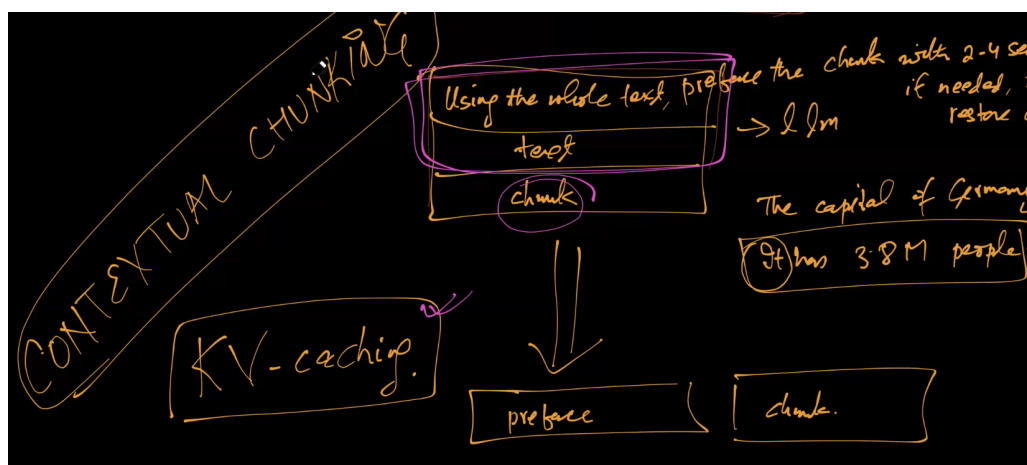


Figure 9: Illustration of Contextual Chunking.

The problem is that small chunks lose context. The solution is using the whole text, preface the chunk with 2-4 sentences if needed to restore context. The example shows: original text “The capital of Germany is Berlin. Its population is 3.8 million people.” The chunk is “It has 3.8M people.” The preface generated by LLM restores context, and the augmented chunk becomes “The capital of Germany is Berlin. It has 3.8M people.” The key concept is Contextual Preface Enrichment - prepending brief, model-generated explanations to tiny data segments clarifies ambiguous references while maintaining semantic purity.

Developed as a powerful new genre of chunking by Anthropic¹¹, the solution is blindingly simple in hindsight. You generate small, focused chunks, but you do not store them raw. Instead, you pass the chunk along with the entire surrounding document text to an LLM. You instruct the machine to generate a brief preface of 2 to 4 sentences that restores the missing background context, and then prepend that preface directly to the chunk.

The critical implementation rule is that the LLM must never tamper with or rewrite the original text of the chunk; it only generates the descriptive preface.

Example: If a raw text chunk splits mid-sentence and reads: *“Its population is 3.8 million people.”* (or 3.6 to 3.9 million based on recent data for Berlin), searching for “Berlin” would completely miss this chunk because the word Berlin doesn’t exist in the text. By passing it to an LLM with the wider text, the machine generates a preface: *“The capital of Germany is Berlin.”* The stored chunk becomes: *“The capital of Germany is Berlin. Chunk: Its population is 3.8 million people.”* Now, the pronoun ambiguity is completely resolved.

8.1.3 Optimizing Compute: Determining What Context to Provide

From a pure semantic perspective, passing the entire document as context is ideal because an explicit reference (like “Equation 3.6”) might be sitting four chapters prior. However, sending a massive 50,000-token document to enrich a tiny 50-token chunk adds massive compute costs, soaring token bills, and extreme latency for every single chunk call.

Reducing Context Scope To optimize this process, you can limit the context window provided to the LLM to just the surrounding chapter, section, or a few proximal paragraphs, assuming with high probability that the necessary context lives nearby.

KV Caching To make contextual chunking practical and affordable, you must exploit a hardware server feature called **KV (Key-Value) Caching**. In the transformer self-attention equation

$$Attention = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

every single token projects a Query (to seek its own meaning), a Key, and a Value (to act as a look-up dictionary entry for other tokens).

When processing contextual chunking, the large background document and system prompt remain completely static; only the small 50-token chunk changes with each call. Computing the Key-Value dictionary for 50,000 tokens over and over is computationally expensive. With KV Caching enabled on the inference server, the dictionary for the

¹¹<https://www.anthropic.com/news/contextual-retrieval>

static text is computed exactly once and held in memory. The server only calculates the marginal math for the changing chunk tokens.

According to official pricing tables for models like GPT, Claude, or Gemini, utilizing token caching drops your input token costs to a small fraction of the regular price and makes execution significantly faster. You can easily achieve this by hosting a good local model (like a Llama variant) on your own data center using **vLLM**¹² or **LightLLM**¹³ to gain trivial, ultra-low latency contextual chunking.

8.2 Late Chunking

8.2.1 Core Insight

Late chunking¹⁴ Jin et al. [2024] represents the absolute state-of-the-art for text data. Traditional RAG architectures chunk text early in human language space before creating vectors, which permanently cuts off contextual links. Late chunking completely reverses this sequence: **you embed the entire document first, and defer chunking to the machine space.**

Instead of viewing chunking as physically cutting text, it treats chunking as merely remembering a list of token index boundaries. If an entire document stream has M total tokens (T_1 all the way to T_M), chunking is simply keeping a separate variable array of index locations where the knife drops (e.g., boundary drops at token index 100, 187, and 251).

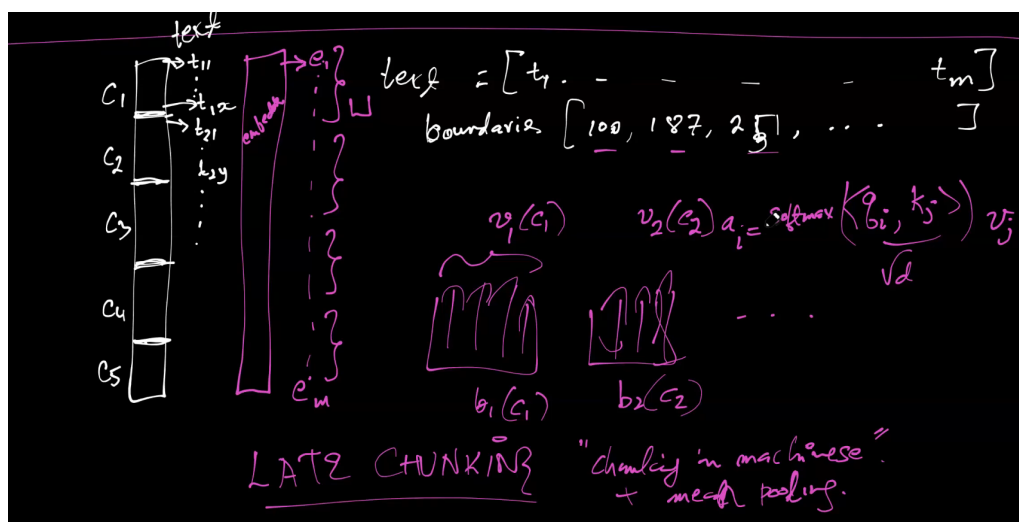


Figure 10: The figure illustrates the Late Chunking workflow.

Workflow of Late Chunking:

¹²<https://github.com/vllm-project/vllm>

¹³<https://github.com/ModelTC/lightllm>

¹⁴<https://jina.ai/news/late-chunking-in-long-context-embedding-models/>

1. **Full Text Embedding:** You take the entire un-cut document text and pass it as a single sequence through the transformer model. Every single token undergoes self-attention, meaning every token vector looks far and wide across the entire document to soak in its full contextual meaning.
2. **Extract Token Vectors:** Instead of applying traditional pooling (like mean pooling the whole text, CLS pooling, or grabbing the last token) to output a single document vector, you stop the model right before the final pooling step. If the document has 1,000 tokens, the machine outputs 1,000 individual contextualized vectors (e_1 all the way to e_m).
3. **Bucketize by Index:** Look at your saved variable of chunk boundaries. Group those highly contextualized token vectors into their respective chunk buckets (b_1 , b_2 , etc.) matching your original index locations.
4. **Mean Pooling:** Apply mean pooling (mathematically averaging all the vector arrows together) inside each separate bucket to compress them. This outputs a single final vector, v_i , for each chunk.

8.2.2 Why It Works

Because the chunking occurs *after* the text has passed through the transformer's self-attention mechanism, every single token vector has already interacted with every other token in the entire document.

Example: If a chunk contains the text “*Its population is 3.8 million people,*” the token vector for “Its” has already looked across the document, identified “Berlin,” and deeply soaked that contextual information into its mathematical representation. When you mean pool that bucket, the resulting vector inherently “knows” it represents Berlin. If a user queries for the population of Berlin, this chunk vector will easily bubble up as the nearest neighbor in a K-Nearest Neighbor (KNN) vector search.

Comparison of Chunking in Plain-text English and in *Machine*.

Attribute	Chunking in English (Human Space)	Chunking in <i>Machine</i> (Late Chunking)
Context	Completely broken and isolated at boundaries	Fully preserved across the entire text stream
Pronouns	Ambiguous and unlinked (e.g., “its”, “this”)	Mathematically linked via self-attention layers
Passes	Requires N separate passes for N chunks	Single forward pass through the transformer

Questions for Pondering

- What is the core mechanism of Contextual Preface Enrichment?
- How does Late Chunking mathematically preserve context better than traditional RAG?

9 Shifting Paradigms

9.1 Visual Embeddings

9.1.1 *The Concept*

Instead of parsing a PDF into text chunks, you completely abandon text parsing. You convert every page of a document into a standalone image (PNG) and pass those images directly into a **Vision-Language Model (VLM)** to generate page-level visual embeddings.

When testing basic vision models like **CLIP** or **SigLIP** for this task, the retrieval quality is incredibly poor—traditional text embeddings easily outperform them. However, when you use a highly advanced vision-language understanding model like **Qwen 3 Vision**¹⁵ model, the results become exceptionally sharp and precise.

9.1.2 *When Visual Embedding Works Best*

This alternative paradigm is heavily superior for complex scientific, technical, and engineering documents (like Bishop’s PRML). It completely bypasses parsing data loss by keeping the page layout completely whole. It naturally captures tables, distribution plots, complex mathematical formulas, handwritten notes, and spatial headers that traditional text parsers destroy.

9.1.3 *The DeepSeek OCR Paper*

A massive breakthrough occurred with a research paper called **DeepSeek OCR** [DeepSeek-AI \[2025\]](#). Despite the name, it is not an OCR tool at all. Instead of expanding the context window of an LLM by feeding it raw text tokens, they did something completely unorthodox: **they treated massive walls of text as vision tokens**.

They compressed vast streams of text into a highly dense visual representation, passed it through an encoder with convolutional layers, and handed a dense latent visual embedding to the LLM alongside the user’s query. Because images are incredibly dense

¹⁵<https://github.com/QwenLM/Qwen-VL>

information containers, they managed to compress massive context windows into fewer vision tokens, achieving extraordinary accuracy and speed.

9.1.4 *Vision Language Model Demonstration*

The immense power of advanced VLMs was demonstrated in a live classroom experiment using a cryptic, handwritten mathematical formula on a whiteboard:

$$\partial^2 = 0 \tag{1}$$

Without providing any textual hints or context, the VLM achieved full vision-language understanding:

1. It perfectly transcribed the messy handwriting of the partial derivative symbol.
2. It linked the image to its internal world knowledge of differential geometry.
3. It correctly explained the deep mathematical fact: **“The boundary of a boundary is zero.”**

To explain this visually: the solid core sphere of the Earth has a surface, which acts as its boundary. However, if you are an ant walking along that 2D surface, you can walk forever in any direction without ever hitting an edge—meaning the boundary itself has no boundary. Similarly, a metallic singing bowl has a physical boundary called the rim, but walking along the circular rim reveals no end point. The VLM successfully decoded this abstract concept purely from a handwritten image.

9.1.5 *Visual Chunking for Video*

When dealing with video data, visual chunking shifts boundaries:

- You can treat every individual video frame frame-by-frame as a standalone image chunk.
- You can group a set number of sequential frames together into a single chunk.

Dividing a static image into smaller visual patches and storing patch embeddings is a terrible practice because vector databases should never return isolated image patches in search results. For video, the active research question is determining the exact boundary layout for how many frames put together forms a semantically meaningful unit.

Language models are rapidly moving past purely word-based linguistic tokens. The absolute cutting edge of AI architecture is happening deeply within the multimodal domain (images, video, voice, and audio). Vision-language models are proving heavily superior for complex structural tasks.

Questions for Pondering

- How did the DeepSeek OCR paper fundamentally redefine context ingestion?
- Why do traditional basic vision models (like CLIP) fail at document retrieval compared to advanced VLMs?

10 Key Takeaways

- **The Chunking Dilemma:** Text chunking forces a trade-off between semantic purity (small chunks) and contextual integrity (large chunks), often causing issues like the Centroid Delusion or broken endophora.
- **One Size Does Not Fit All:** Hardcoding chunk sizes (e.g., 512 tokens) is an architectural flaw; distinct document genres necessitate specialized chunking strategies.
- **Contextual and Late Chunking:** State-of-the-art methods like Contextual Preface Enrichment and Late Chunking preserve semantic meaning by retaining or restoring surrounding context.
- **Visual RAG for Complex Layouts:** For technical documents, embedding entire pages as images using Vision-Language Models (VLMs) captures spatial, tabular, and graphical context that text parsers destroy.
- **Maintain Multiple Indices:** Running parallel search indices (e.g., text, visual, dense, sparse) and merging results via re-ranking provides a massive retrieval performance boost.
- **Embedding Fragility:** Embedding models remain highly sensitive to minor structural and grammatical changes; they must be deployed and evaluated with rigorous caution.

References

- Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- DeepSeek-AI. Deepseek-ocr: Contexts optical compression. *arXiv preprint arXiv:2510.18234*, 2025.
- DeepSeek-AI. Conditional memory via scalable lookup: A new axis of sparsity for large language models. <https://github.com/deepseek-ai/Engram>, 2026.
- Fyodor Dostoevsky. *Crime and Punishment*. The Russian Messenger, 1866.

Qiao Jin et al. Late chunking: Contextual chunk embeddings using long-context embedding models. *arXiv preprint arXiv:2409.04701*, 2024.

Brian W Kernighan and Dennis M Ritchie. *The C Programming Language*. Prentice Hall, 1988.

Note: This document is a summary of Week 3 of the Beyond RAG: Advanced Agentic Knowledge Architectures course.
For complete details, refer to the course portal, lab notebooks, and session recordings.